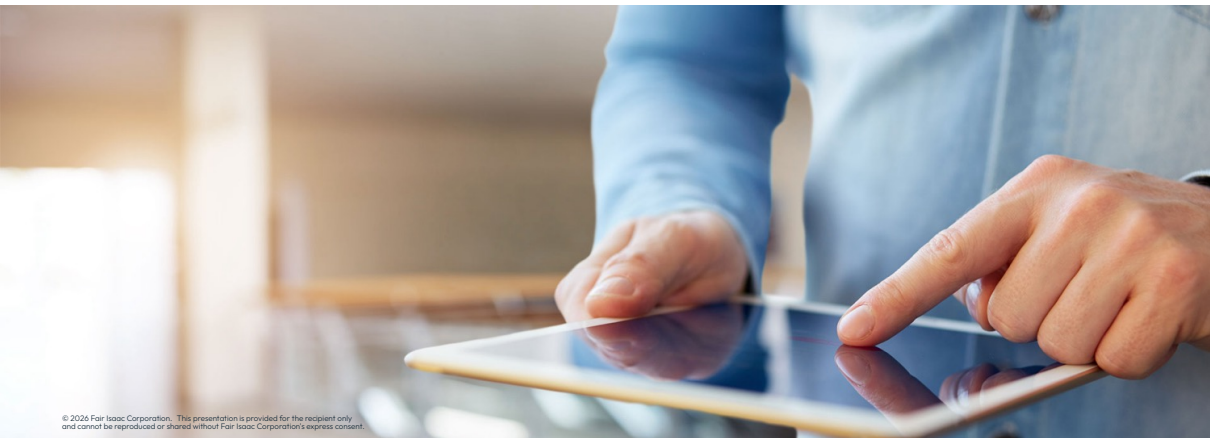


Nonlinear solving

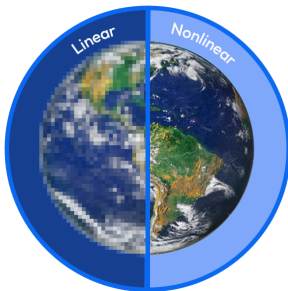




Nonlinear Optimization (incl.
FICO[®] Xpress Global)

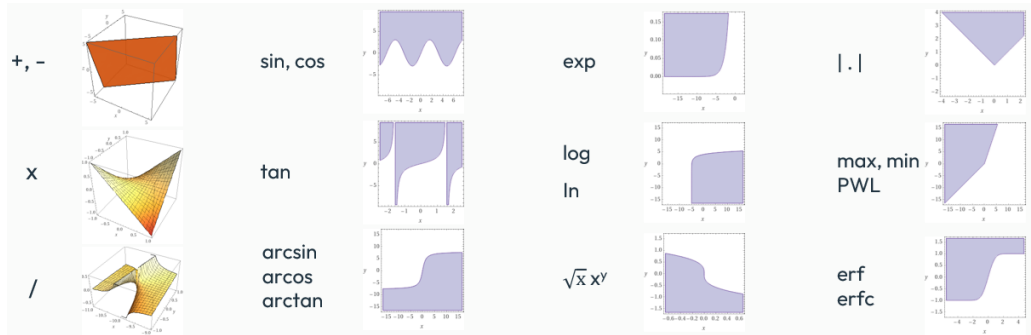
The world is nonlinear

- Many applications are inherently nonlinear:
 - Chemical processing, electrical engineering, gas transmission,...
- *A priori* linearization can work in some cases and is often quite fast...
 - ...but can be arbitrarily bad w.r.t. both feasibility and optimality.
- In many applications, guaranteed global optima are required
 - Small objective difference can equate to huge financial loss or regulatory non-compliance



What kind of nonlinear functions does FICO® Xpress support?

- All combinations of:



- Examples:

$$\sum_{j=1}^n e^{x_j} x_j^3$$

$$x_1^3 - 3x_1^2 + 3x_1 - 1$$

$$x_1 \sin(x_2)$$

$$\sqrt{\sum_{j=1}^n x_j^2}$$

$$x \sin\left(\frac{1}{x}\right)$$

$$\prod_{j=1}^n \log(x_j)$$

Modeling nonlinear problems in Python

- **Nonlinear problems**, i.e. problems containing **at least one nonlinear constraint or objective**, can be modeled via the Xpress Python interface:
 - Nonlinear expressions follow the **same relational and arithmetic logic** as linear expressions
 - Available arithmetic operators: `+`, `-`, `*`, `/`, `**` (which is the Python equivalent for the power operator, `^`)
 - **Univariate functions** can be used from the following list: `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `exp`, `log`, `log10`, `abs`, `sign`, `sqrt`, `erf`, and `erfc`
 - The **multivariate functions** `min` and `max` can receive an arbitrary number of arguments
 - Piecewise linear functions (`pwl`) are discussed later

Modeling nonlinear problems in Python

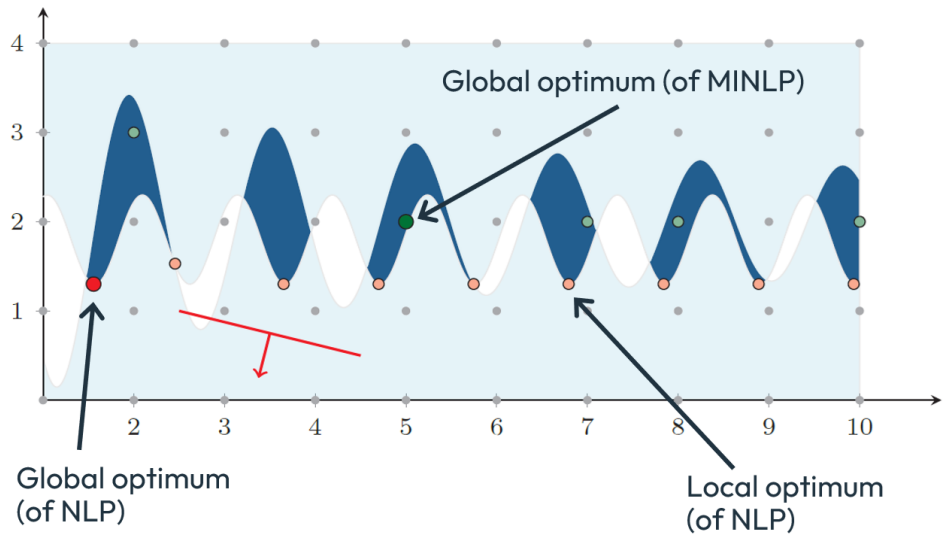
- Examples of nonlinear problem elements:

```
p.addConstraint(x**4 + 2 * x**2 - 5 >= 0) # polynomial constraint
p.addConstraint(xp.sin(math.pi * x) == 0) # terrible way to constrain x to be integer
p.addConstraint(x**2 * xp.sign(x) <= 4) # signum function
p.setObjective((a-x)**2 + b*(y-x**2)**2) # minimize Rosenbrock function
```



Finding help: For more information about modeling nonlinear problems, browse the *FICO® Xpress NonLinear reference manual*

Local vs Global Optimization



Solvers

- FICO® Xpress Optimizer:
 - Used to solve **convex quadratic problems** (QP, MIQP, QCQP, SOCP)
- FICO® Xpress Global:
 - Finds a global optimum, including valid bounds and gaps during the solve
 - Uses spatial branching, convexification cuts, and local solver heuristics
 - Detailed techniques covered in upcoming slides
- FICO® Xpress NonLinear = Local solvers (find **local optima**):
 - SLP: *Successive Linear Programming*
 - Iterative linear approximations
 - Nonlinear Branch&Bound (MISLP)
 - Artelys Knitro: used as a plugin to solve highly nonlinear models. Some algorithms include:
 - Interior Point/direct (default)
 - Augmented Lagrangian
 - Nonlinear Branch&Bound
 - ...

Convex quadratic problems (QP, MIQP, MIQCQP)

- **Quadratic Programming (QP)**: quadratic objective, linear constraints
 - Solved by Xpress Optimizer using KKT-Simplex (default) or Newton-Barrier algorithm
- **Mixed Integer Quadratic Programming (MIQP)**: QP with integer variables
 - Solved by Xpress Optimizer using KKT-Simplex-based Branch-and-Bound (default)
 - KKT-Simplex solves QP relaxations at each node

Convex quadratic problems (QP, MIQP, MIQCQP)

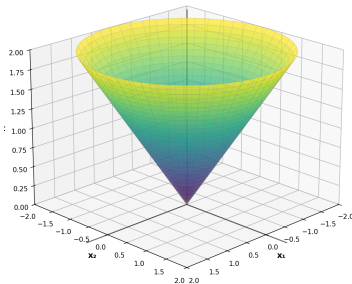
- **Quadratic Programming (QP)**: quadratic objective, linear constraints
 - Solved by Xpress Optimizer using KKT-Simplex (default) or Newton-Barrier algorithm
- **Mixed Integer Quadratic Programming (MIQP)**: QP with integer variables
 - Solved by Xpress Optimizer using KKT-Simplex-based Branch-and-Bound (default)
 - KKT-Simplex solves QP relaxations at each node
- **Quadratically Constrained QP (QCQP)**: quadratic terms in constraints
 - Handled by Xpress Optimizer using Barrier-based Branch-and-Bound (default)
- **Mixed Integer Quadratically Constrained QP (MIQCQP)**: QCQP with integer variables
 - Solved by Xpress Optimizer using Outer Approximation (default)

Convex quadratic problems (QP, MIQP, MIQCQP)

- **Quadratic Programming (QP)**: quadratic objective, linear constraints
 - Solved by Xpress Optimizer using KKT-Simplex (default) or Newton-Barrier algorithm
- **Mixed Integer Quadratic Programming (MIQP)**: QP with integer variables
 - Solved by Xpress Optimizer using KKT-Simplex-based Branch-and-Bound (default)
 - KKT-Simplex solves QP relaxations at each node
- **Quadratically Constrained QP (QCQP)**: quadratic terms in constraints
 - Handled by Xpress Optimizer using Barrier-based Branch-and-Bound (default)
- **Mixed Integer Quadratically Constrained QP (MIQCQP)**: QCQP with integer variables
 - Solved by Xpress Optimizer using Outer Approximation (default)
- **Solving these problems does not require an Xpress Nonlinear/Global solver license**

Second-order cone programming (SOCP, MISOCP)

- **Second-Order Cone Programming (SOCP):** Convex optimization with conic constraints
 - Special form of convex nonlinear constraints: $\|Ax + b\| \leq c^T x + d$, linear objective function
 - Applications: Portfolio optimization, robust optimization, signal processing



- **Mixed Integer SOCP (MISOCP):** SOCP with integer variables
 - Common in portfolio optimization with cardinality constraints

MIQCQP and MISOCP solution approaches

- **KKT-Simplex Branch-and-Bound**: Primary method for convex MIQP (quadratics only in objective)
 - Solves KKT optimality conditions using Simplex-like approach
 - Handles complementarity condition (zero/non-zero variables) similar to variable bounds in Simplex

MIQCQP and MISOCP solution approaches

- **KKT-Simplex Branch-and-Bound**: Primary method for convex MIQP (quadratics only in objective)
 - Solves KKT optimality conditions using Simplex-like approach
 - Handles complementarity condition (zero/non-zero variables) similar to variable bounds in Simplex
- **Outer Approximation**: Primary method for convex MIQCQP/MISOCP
 - Approximates quadratic/conic constraints with supporting hyperplanes
 - Iteratively refines linear approximation

MIQCQP and MISOCP solution approaches

- **KKT-Simplex Branch-and-Bound**: Primary method for convex MIQP (quadratics only in objective)
 - Solves KKT optimality conditions using Simplex-like approach
 - Handles complementarity condition (zero/non-zero variables) similar to variable bounds in Simplex
- **Outer Approximation**: Primary method for convex MIQCQP/MISOCP
 - Approximates quadratic/conic constraints with supporting hyperplanes
 - Iteratively refines linear approximation
- **Barrier-based Branch-and-Bound**: Alternative for MIQCQP/MISOCP
 - Uses Newton-Barrier algorithm to solve continuous relaxations

MIQCQP and MISOCP solution approaches

- **KKT-Simplex Branch-and-Bound**: Primary method for convex MIQP (quadratics only in objective)
 - Solves KKT optimality conditions using Simplex-like approach
 - Handles complementarity condition (zero/non-zero variables) similar to variable bounds in Simplex
- **Outer Approximation**: Primary method for convex MIQCQP/MISOCP
 - Approximates quadratic/conic constraints with supporting hyperplanes
 - Iteratively refines linear approximation
- **Barrier-based Branch-and-Bound**: Alternative for MIQCQP/MISOCP
 - Uses Newton-Barrier algorithm to solve continuous relaxations
- All methods: Branch on integer variables to explore solution space

(MI)SOCP/QCQP/QP tuning

- For quadratically constrained problems (QCQP) and second-order cone programs (SOCP):
 - **miqcpalg**: Chooses between nonlinear Branch&Bound with barrier solves and an LP-based outer approximation
 - On most instances outer approximation performs better
 - In some cases barrier can significantly outperform it

(MI)SOCP/QCQP/QP tuning

- For quadratically constrained problems (QCQP) and second-order cone programs (SOCP):
 - **miqcpalg**: Chooses between nonlinear Branch&Bound with barrier solves and an LP-based outer approximation
 - On most instances outer approximation performs better
 - In some cases barrier can significantly outperform it
- Preprocessing controls for quadratic problems:
 - **preconvertobjtocons**: Whether to keep quadratic objective and solve with KKT-simplex or move it to a constraint to solve with barrier or OA
 - **preconvertseparable**: Whether to keep SCOP as is or convert to many smaller or even diagonal cones

Python Notebook [PN-4.1]: Circle packing - local vs global solver

- Navigate to [Exercise 4.1 - Local vs Global NLP solver](#) in the notebook
- The problem: pack N circles inside a unit square to [maximize the sum of radii](#)
- Run the local solver (SLP) code cell and note the solution quality
- Run the global solver code cell and compare:
 - Are the solutions the same?
 - What are the differences in objective value and solving time?
- Run the visualization cell to compare both solutions side by side

Xpress NonLinear vs Xpress Global

Xpress NonLinear

Local optimum

No bounds on distance to optimum

No proof of infeasibility

Fast in general

Supports user functions

Xpress Global

Global optimum

Duality/Optimality Gap

Can detect infeasibility, required for nonlinear IIS

Possibly long branch-and-bound search

No user functions

When to use FICO® Xpress Global ?

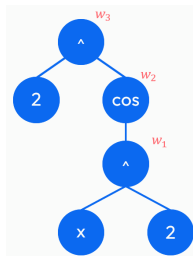
- Whenever a reliable bound on the optimal value is needed:
 - Solution quality
 - Regulatory compliance
 - Workflow validation
- Problems with many local optima
- Nonlinear problems with discrete entities
- Infeasibility detection/guarantee: IIS, repairinfeas

Xpress Global solver: High-level overview

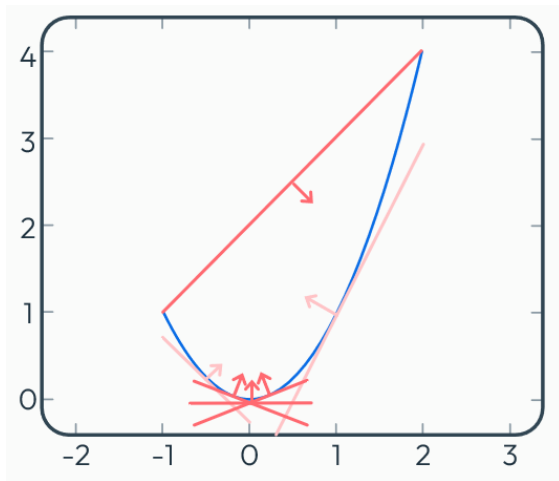
- The Global solver finds provably optimal solutions through:
 1. **Nonlinear presolve**: Bound tightening and problem simplification
 2. **Reformulation**: Transform problem using auxiliary variables (DAG representation)
 3. **MIP presolve**: Standard presolve on reformulated problem
 4. **Root LP solve**: Solve initial relaxation with convexification cuts
 - If unbounded, use bounding box approach
 5. **Branch-and-bound search**:
 - Additional propagation and convexification cuts to enforce nonlinearities
 - Branch on both integer and continuous variables (spatial branching)
 - Run local NLP solver on MIP-feasible solutions as a heuristic

Reformulation via Directed Acyclic Graph

- **Example:** $2^{\cos(x^2)} \leq 1$
- Build the so-called expression tree
- Identify common subtrees to build a **Directed Acyclic Graph**
- Introduce an auxiliary variable for each internal node:
 - $w_3 \leq 1$ [$w_1 = x^2$, $w_2 = \cos(w_1)$, $w_3 = 2^{w_2}$]
- Identify and propagate domain reductions:
 - $w_3 \leq 1 \Rightarrow w_2 = \log_2(w_3) \leq \log_2(1) = 0$
- Ignore the nonlinear part initially and solve for the linear system



Convexification cuts



Convexification cuts

- Global solver uses **convexification cuts** to create linear over/underestimators:
 - Cuts are **added dynamically** at three stages:
 - Initial root relaxation setup
 - Root and node cutloops
 - During branching evaluation
 - **Standard cuts** for most operators:
 - Secant cuts: Linear interpolation between bounds
 - Tangent cuts: First-order approximations
 - **Additional specialized cuts** for:
 - Quadratics: Lifted tangent (LTI) cuts, RLT cuts, SDP cuts
 - Trigonometric functions: Specialized approximations
- These cuts progressively tighten the relaxation as the tree search proceeds

Spatial branching strategy

- **Spatial branching** extends MIP branching to continuous variables:
 - Branch on continuous variables to partition nonlinear terms' domains
 - Enables tighter convexification cuts in sub-problems

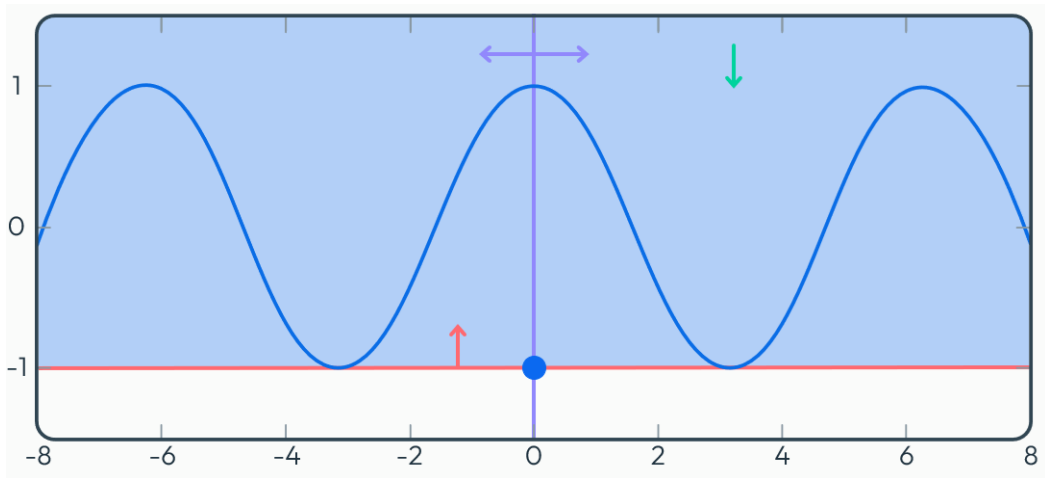
Spatial branching strategy

- **Spatial branching** extends MIP branching to continuous variables:
 - Branch on continuous variables to partition nonlinear terms' domains
 - Enables tighter convexification cuts in sub-problems
- **Branch candidate evaluation:**
 - Uses standard MIP technology: strong branching and pseudocosts
 - Propagates and cuts from candidates before evaluation
 - No inherent preference between integer and continuous variables
 - By default prefers original over auxiliary variables
 - *Exception:* When auxiliary branching leads to many bound changes at root

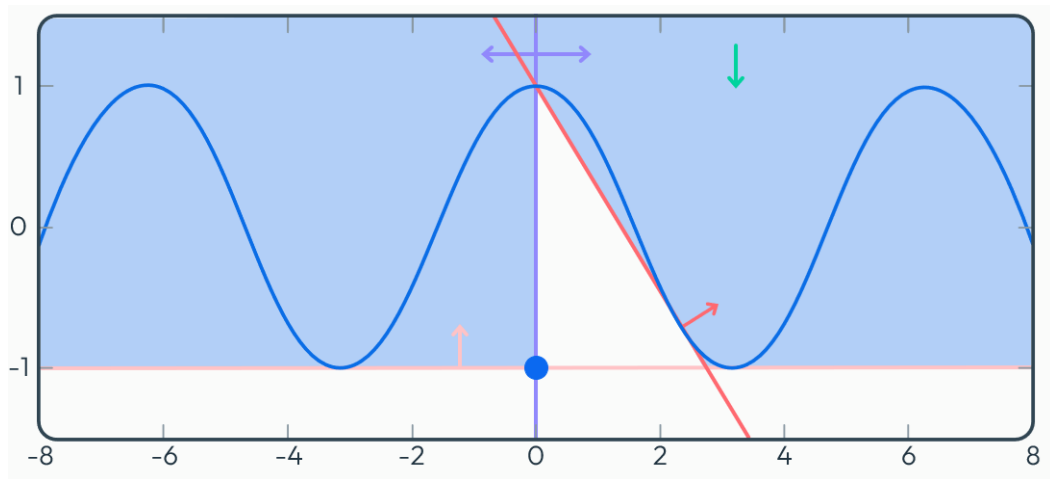
Spatial branching strategy

- **Spatial branching** extends MIP branching to continuous variables:
 - Branch on continuous variables to partition nonlinear terms' domains
 - Enables tighter convexification cuts in sub-problems
- **Branch candidate evaluation:**
 - Uses standard MIP technology: strong branching and pseudocosts
 - Propagates and cuts from candidates before evaluation
 - No inherent preference between integer and continuous variables
 - By default prefers original over auxiliary variables
 - *Exception:* When auxiliary branching leads to many bound changes at root
- **Heuristics integration:**
 - MIP heuristics run normally during search
 - When MIP-feasible but NLP-infeasible solution is found, local NLP solver fixes integers and solves reduced problem
 - Uses MIP-feasible solution as initial values for NLP

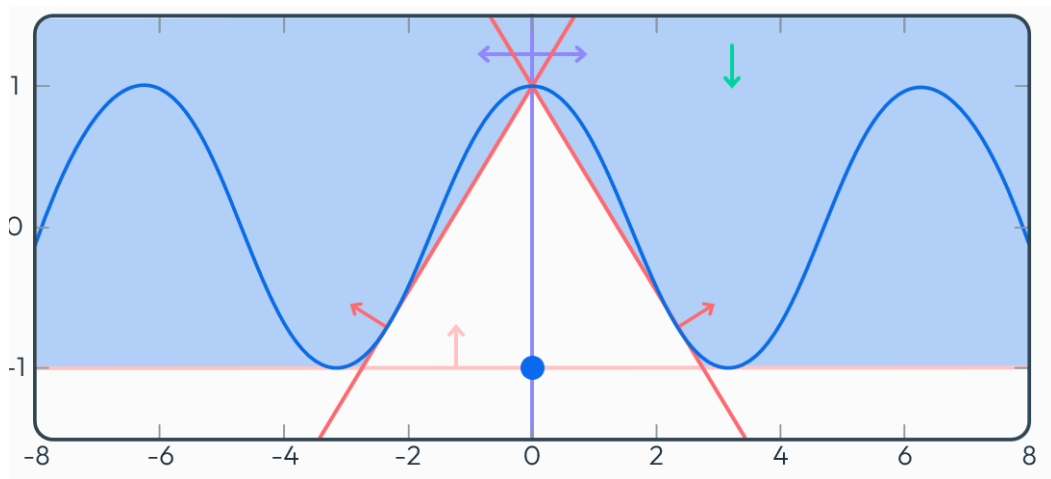
Convexification Cuts & Spatial Branching



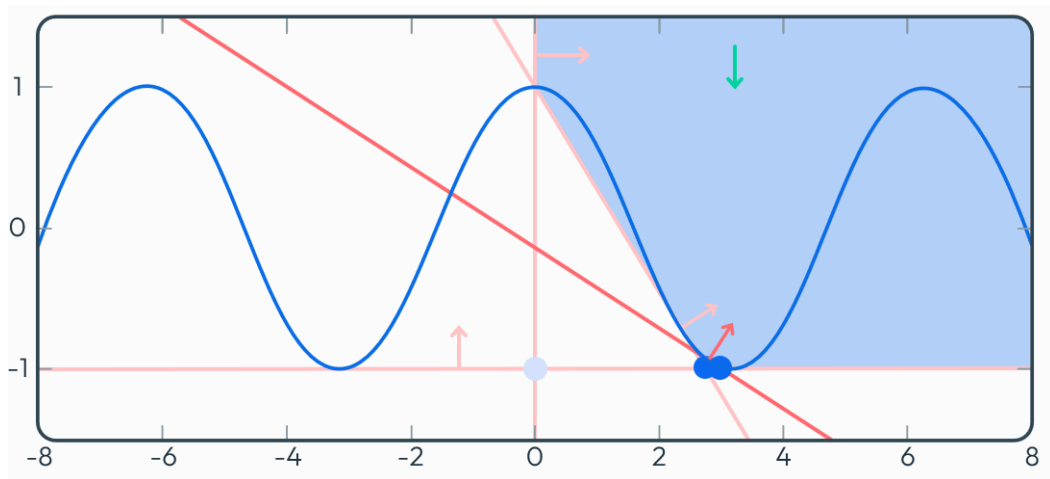
Convexification Cuts & Spatial Branching



Convexification Cuts & Spatial Branching



Convexification Cuts & Spatial Branching



Bounding Box

- LP with auxiliary variables **can be unbounded**:
 - $\min \{x \mid x^2 - x^4 \geq 0\}$
- Apply initial bound propagation and convexification cuts.
- Solve the initial LP relaxation, if unbounded:
 - Impose artificial bounds on the original variables and resolve.
 - If still unbounded, impose bounds on the auxiliary variables.

Bounding Box

- If bounding box was applied:
 - Xpress Global finds (globally) best solution **within those bounds**
 - Better solutions may exist outside those bounds
 - **Solstatus** will be **Feasible** instead of **Optimal**.
- We cannot (and do not) declare global optimality in this case
- Size of bounding box can be adjusted using control **globalboundingbox**.
- **Importance of tight bounds:**
 - Xpress Global **requires bounds** on variables to function effectively
 - Tighter bounds on more variables lead to better performance and stronger cuts
 - Weak bounds result in weaker convexification cuts and slower solves or worse dual bounds

Nonlinear solver selection and tuning controls

- Xpress provides various controls to tune nonlinear solver performance
 - Reasonable defaults are preconfigured, but tuning can significantly improve performance for specific problem types
- Solver selection controls to choose between local and global solvers:
 - `nlp solver`: Switches between local and global solves
 - `local solver`: Switches between different local solvers (SLP or Knitro)

Nonlinear solver selection and tuning controls

- Xpress provides various controls to tune nonlinear solver performance
 - Reasonable defaults are preconfigured, but tuning can significantly improve performance for specific problem types
- **Solver selection controls** to choose between local and global solvers:
 - **nlp solver**: Switches between local and global solves
 - **local solver**: Switches between different local solvers (SLP or Knitro)
- **Xpress Tuner** supports all kinds of nonlinear solves:
 - Set **tunermethod** accordingly:
 - 6: SLP
 - 7: MISLP
 - 9: Global



Note: *Important caution for SLP tuning: Tuner optimizes for faster solves, which can favor settings that quickly converge to poor local optima over settings that find better solutions more slowly. Always manually evaluate solution quality vs. solve time trade-offs when using tuner with local solvers*

Global solver tuning

- Cutting plane controls for convexification:
 - `globalnuminitnlpcuts`: How many cuts to separate and add to the root LP for each nonlinear term
 - More cuts improve the root bound and may help with unboundedness
 - But will make LP solves more expensive
 - `globalnlpcuts`, `globaltreenlpcuts`: How many rounds of convexification cuts to add at each node before switching to branching

Global solver tuning

- **Cutting plane controls** for convexification:
 - **globalnuminitnlpcuts**: How many cuts to separate and add to the root LP for each nonlinear term
 - More cuts improve the root bound and may help with unboundedness
 - But will make LP solves more expensive
 - **globalnlpcuts**, **globaltreenlpcuts**: How many rounds of convexification cuts to add at each node before switching to branching
- **Branching and propagation controls**:
 - **globalspatialbranchifpreferorig**: Whether to always branch on original columns (1) or consider both original and auxiliary columns (0)
 - **globalspatialbranchcuttingeffort**, **globalspatialbranchpropagationeffort**: Control how much effort to spend on cutting and propagation when evaluating branching entities



Note: *Propagation and cutting on branching entities are important for good decisions but can be time and memory intensive*

Global solver tuning

- Heuristic controls:
 - `globalsheurstrategy`: How aggressively to run SLP on integer-feasible node solutions

Global solver tuning

- **Heuristic controls:**
 - **globalsheurstrategy**: How aggressively to run SLP on integer-feasible node solutions
- **Bound tightening:**
 - **globalpresolveobb**: Whether to run optimization-based bound tightening at the root
 - Off by default
 - Very costly (up to two LP solves per column)
 - Can improve bounds and lead to stronger cuts

Global solver tuning

- **Heuristic controls:**
 - **globalsheurstrategy**: How aggressively to run SLP on integer-feasible node solutions
- **Bound tightening:**
 - **globalpresolveobbt**: Whether to run optimization-based bound tightening at the root
 - Off by default
 - Very costly (up to two LP solves per column)
 - Can improve bounds and lead to stronger cuts
- **Example of setting global solver controls:**

```
p.controls.globalnuminitnlpcuts = 10  
p.controls.globalnlpcuts = 5
```

```
# add 10 cuts per NL term at root  
# 5 rounds of cuts per node
```

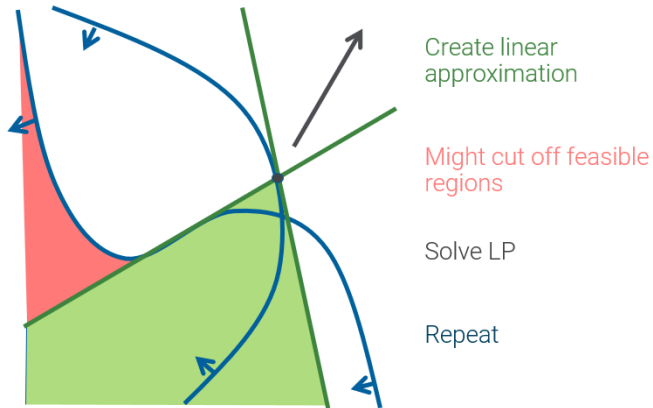
Successive Linear Programming (SLP)

- **SLP algorithm:** Iteratively solves linear approximations of the nonlinear problem
 - At each iteration, linearize nonlinear terms around current solution by introducing **delta-variables** and **penalty variables**:

$$\begin{array}{ll} \min c^T x & \min c^T x + p^T z \\ \text{s.t. } g_j(x) \leq 0 \quad \forall j \leq m & \text{s.t. } g_j(x_0) + \nabla g_j(x_0)^T y - z_k \leq 0 \quad \forall j \leq m \\ & x = x_0 + y \\ & -S \leq y \leq S \\ & z \geq 0 \end{array}$$

- Solve the resulting LP to find a new iterate
- Repeat until convergence (variables and objective stabilize)

Successive Linear Programming (SLP)



SLP: Trust regions and penalty costs

- **Trust regions (step bounds):** Delta-variables are step-bounded with dynamic updates
 - Ensures LP solution stays in neighborhood of current point where linearization is valid
 - Prevents unbounded linearizations when approximation is poor
 - Step bounds adapt during solve based on progress

SLP: Trust regions and penalty costs

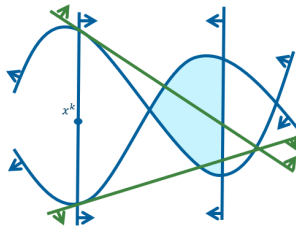
- **Trust regions (step bounds):** Delta-variables are step-bounded with dynamic updates
 - Ensures LP solution stays in neighborhood of current point where linearization is valid
 - Prevents unbounded linearizations when approximation is poor
 - Step bounds adapt during solve based on progress
- **Penalty costs:** Penalize constraint violations during iterations
 - Allows temporary infeasibility to escape poor regions
 - Fix penalty breakers if penalty terms become too large
 - Costs adjusted automatically to balance feasibility and objective improvement

SLP: Trust regions and penalty costs

- **Trust regions (step bounds):** Delta-variables are step-bounded with dynamic updates
 - Ensures LP solution stays in neighborhood of current point where linearization is valid
 - Prevents unbounded linearizations when approximation is poor
 - Step bounds adapt during solve based on progress
- **Penalty costs:** Penalize constraint violations during iterations
 - Allows temporary infeasibility to escape poor regions
 - Fix penalty breakers if penalty terms become too large
 - Costs adjusted automatically to balance feasibility and objective improvement
- Solve KKT subproblem to check for local optimality

SLP: Infeasibility

- Non-convex non-linear with feasible region.
- Iterate solution x^k in infeasible region.
- Can result in infeasible LP
- Use penalty variables to make LP feasible.



SLP convergence and local optima

- **Convergence criteria:** SLP stops when solution stabilizes
 - **Strict convergence:** Variable values don't change significantly
 - **Extended convergence:** Objective value and feasibility stabilize
 - Multiple tolerance levels for different convergence stages

SLP convergence and local optima

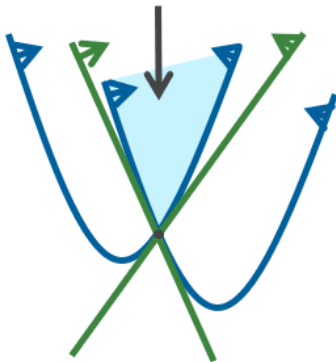
- **Convergence criteria:** SLP stops when solution stabilizes
 - **Strict convergence:** Variable values don't change significantly
 - **Extended convergence:** Objective value and feasibility stabilize
 - Multiple tolerance levels for different convergence stages
- **Local optima behavior:** SLP finds local optima, not necessarily global
 - Smaller penalty costs and larger step bounds allow exploring bigger parts of the feasible region and potentially finding better local optima at the cost of slower convergence
 - Initial point significantly affects which local optimum is found
 - For problems with many local optima, use multistart or Xpress Global

SLP convergence and local optima

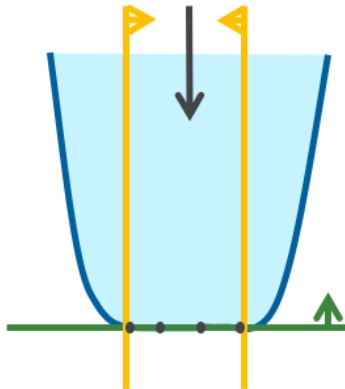
- **Convergence criteria:** SLP stops when solution stabilizes
 - **Strict convergence:** Variable values don't change significantly
 - **Extended convergence:** Objective value and feasibility stabilize
 - Multiple tolerance levels for different convergence stages
- **Local optima behavior:** SLP finds local optima, not necessarily global
 - Smaller penalty costs and larger step bounds allow exploring bigger parts of the feasible region and potentially finding better local optima at the cost of slower convergence
 - Initial point significantly affects which local optimum is found
 - For problems with many local optima, use multistart or Xpress Global
- **MISLP (Mixed-Integer SLP):** Combines SLP with Branch-and-Bound
 - Three approaches: Solve NLP then round, MIP outer approximation, or nonlinear Branch&Bound
 - Only guarantees local optimality for given integer solution

What convergence looks like

Strict convergence



Extended convergence



SLP tuning: Penalty costs

- Penalty costs for constraint violations during iterations:
 - **slpobjtpenaltycost**: Estimate initial penalty costs based on factor to objective coefficients
 - Typically values between 1 and 100
 - Smaller values lead to more iterations
 - Higher values tend to find local optima very close to the initial point
 - **slperrorcostfactor**: Factor by which to increase penalty costs in each iteration when rows are still infeasible
 - Typically values between 1 and 5
 - Similar effect to slpobjtpenaltycost, but applied more gradually

SLP tuning: Penalty costs

- Penalty costs for constraint violations during iterations:
 - **slpobjtpenaltycost**: Estimate initial penalty costs based on factor to objective coefficients
 - Typically values between 1 and 100
 - Smaller values lead to more iterations
 - Higher values tend to find local optima very close to the initial point
 - **slperrorcostfactor**: Factor by which to increase penalty costs in each iteration when rows are still infeasible
 - Typically values between 1 and 5
 - Similar effect to slpobjtpenaltycost, but applied more gradually
- Example:

```
p.controls.nlpsolver = 1           # use local solver
p.controls.localsolver = 0        # use SLP
p.controls.slpobjtpenaltycost = 10 # initial penalty factor
p.controls.slperrorcostfactor = 2 # penalty increase factor
```

SLP tuning: Convergence and reformulation

- **Convergence controls:**
 - **slpconvergenceops:** What kind of convergence to require before checking for KKT conditions
 - 3: strict convergence
 - 15: extended convergence
 - 511: check KKT when not progressing any more
 - Can be useful if checking KKT conditions is particularly expensive
 - **slpfilter:** By default SLP returns best solution encountered; unset bit 0 to always get solution from final iterate

SLP tuning: Convergence and reformulation

- **Convergence controls:**
 - **slpconvergenceops**: What kind of convergence to require before checking for KKT conditions
 - 3: strict convergence
 - 15: extended convergence
 - 511: check KKT when not progressing any more
 - Can be useful if checking KKT conditions is particularly expensive
 - **slpfilter**: By default SLP returns best solution encountered; unset bit 0 to always get solution from final iterate
- **Reformulation controls:**
 - **nlpreformulate**: Decides whether min/max/abs/pwl should be handled as MIP constructs or nonlinear
 - Handling them in SLP is usually faster but finds local optima instead of global ones
 - By default converts to MIP if problem becomes fully MIP

SLP tuning: Probing and advanced options

- Probing and bound tightening:
 - **nlpprobing**: Apply bounds on columns and see if we can show infeasibility through propagation
 - Values from 0 to 5 with increasing levels of probing
 - Decrease if it takes too long
 - Increase if running into unboundedness

SLP tuning: Probing and advanced options

- **Probing and bound tightening:**
 - **nlpprobing**: Apply bounds on columns and see if we can show infeasibility through propagation
 - Values from 0 to 5 with increasing levels of probing
 - Decrease if it takes too long
 - Increase if running into unboundedness
- **Advanced iteration controls:**
 - **slpdeltazlimit**: For how many iterations to keep small but nonzero derivatives
 - Can be useful for avoiding bad local minima
 - Makes LP solves harder

Multistart for local solvers

- Both SLP and Knitro can use **multistart** with different initial points to find multiple local optima:
 - **problem.msAddPreset**: Create a given number of random initial points to start SLP from
 - Use `xprs.MSSET_INITIALVALUES` preset
 - **problem.msAddJob**: Manually add individual jobs to the multistart queue with handcrafted initial points or settings

Multistart for local solvers

- Both SLP and Knitro can use **multistart** with different initial points to find multiple local optima:
 - **problem.msAddPreset**: Create a given number of random initial points to start SLP from
 - Use `xprs.MSSET_INITIALVALUES` preset
 - **problem.msAddJob**: Manually add individual jobs to the multistart queue with handcrafted initial points or settings
- Example with multistart:

```
import xpress as xp

p = xp.problem()
# ... define model ...

# add 10 random initial points for multistart
p.msAddPreset(xp.constants.MSSET_INITIALVALUES, 10)

p.optimize()
```

User functions

- A **user function** enables the creation of an expression, the value of which is **computed through external code**:
 - Behaves the same as any other mathematical function, used in a formula to calculate the current value of a coefficient
 - Is normally *free-standing* and needs no access to problem or other data apart from what it receives through its argument list

User functions

- A **user function** enables the creation of an expression, the value of which is **computed through external code**:
 - Behaves the same as any other mathematical function, used in a formula to calculate the current value of a coefficient
 - Is normally *free-standing* and needs no access to problem or other data apart from what it receives through its argument list
- Use cases:
 - When an algebraic description is not possible, for example when working with a **simulation or machine learning** model
 - Functions that are not supported by Xpress

User functions

- A **user function** enables the creation of an expression, the value of which is **computed through external code**:
 - Behaves the same as any other mathematical function, used in a formula to calculate the current value of a coefficient
 - Is normally *free-standing* and needs no access to problem or other data apart from what it receives through its argument list
- Use cases:
 - When an algebraic description is not possible, for example when working with a **simulation or machine learning** model
 - Functions that are not supported by Xpress
- Important considerations when using user functions:
 - Be aware of losses in **determinism** and **performance**
 - **Speed considerations**: User function evaluation can dominate solve time, especially if derivatives are computed numerically

User functions

- Any user-defined function can be called within a problem by using the function `xpress.user()`:

```
xp.user(f, a1, a2, ...)
```

- Where `f` represents the user-defined function name and `a1, a2, ...` the necessary arguments, as in the example below:

```
def myfunc(v1, v2, data):  
    model = MLmodel(v1, v2, data)    # mLmodel() defined elsewhere  
    return model.results
```

```
data = readData()    # readData() defined elsewhere  
x, y = p.addVariable(), p.addVariable()  
p.setObjective(xp.user(myfunc, x, y, data))
```

User functions: Derivatives

- **Derivative computation** for user functions and model expressions:
 - **Numeric derivatives:** Used for user functions if derivatives not provided by the user
 - SLP queries $f(x + \varepsilon e_n)$ for all directions if derivatives not provided
 - Can become a significant bottleneck if function evaluation is costly
 - **Symbolic derivatives:** Default for smaller formulas, provides exact derivatives
 - **Automatic differentiation:** Default for larger formulas, efficient and accurate

User functions: Derivatives

- **Derivative computation** for user functions and model expressions:
 - **Numeric derivatives**: Used for user functions if derivatives not provided by the user
 - SLP queries $f(x + \varepsilon e_n)$ for all directions if derivatives not provided
 - Can become a significant bottleneck if function evaluation is costly
 - **Symbolic derivatives**: Default for smaller formulas, provides exact derivatives
 - **Automatic differentiation**: Default for larger formulas, efficient and accurate
- **Providing user function derivatives** can significantly improve performance:
 - Use specialized Python function types to provide derivative information
 - Many machine learning libraries in Python can provide derivatives automatically
 - See [xpress.user](#) documentation for details on providing derivatives

User functions: Derivatives

- **Derivative computation** for user functions and model expressions:
 - **Numeric derivatives**: Used for user functions if derivatives not provided by the user
 - SLP queries $f(x + \varepsilon e_n)$ for all directions if derivatives not provided
 - Can become a significant bottleneck if function evaluation is costly
 - **Symbolic derivatives**: Default for smaller formulas, provides exact derivatives
 - **Automatic differentiation**: Default for larger formulas, efficient and accurate
- **Providing user function derivatives** can significantly improve performance:
 - Use specialized Python function types to provide derivative information
 - Many machine learning libraries in Python can provide derivatives automatically
 - See [xpress.user](#) documentation for details on providing derivatives
- **Control for user function evaluation**:
 - **xslpevaluate**: By default (0) SLP tries to use derivatives to avoid costly re-evaluations if the new iterate is sufficiently close
 - Set to 1 to force reevaluation if userfunction is discontinuous

MISLP tuning

- MISLP (Mixed-Integer SLP) solves problems with both nonlinear and integer variables using three approaches:
 - **SLP-in-MIP** (default): Nonlinear Branch&Bound with SLP solve in every node - most reliable
 - **SLP-MIP-SLP**: Relax integers, solve SLP, fix linearization and solve MIP, fix integers and resolve SLP - fastest but risky
 - **MIP-in-SLP**: Single SLP solve with linearizations as MIPs - faster when nonlinear part dominates
- **slpmipalgorithm**: Controls which MISLP algorithm to use:
 - Set bit 9: run MIP-in-SLP (usually not recommended)
 - Set bit 10: run SLP-MIP-SLP (fastest but most likely to run into local infeasibility or bad local optima)

MISLP tuning

- MISLP (Mixed-Integer SLP) solves problems with both nonlinear and integer variables using three approaches:
 - **SLP-in-MIP** (default): Nonlinear Branch&Bound with SLP solve in every node - most reliable
 - **SLP-MIP-SLP**: Relax integers, solve SLP, fix linearization and solve MIP, fix integers and resolve SLP - fastest but risky
 - **MIP-in-SLP**: Single SLP solve with linearizations as MIPs - faster when nonlinear part dominates
- **slpmipalgorithm**: Controls which MISLP algorithm to use:
 - Set bit 9: run MIP-in-SLP (usually not recommended)
 - Set bit 10: run SLP-MIP-SLP (fastest but most likely to run into local infeasibility or bad local optima)
- **Cutting planes and heuristics**:
 - **slpcutstrategy**: Set to -1/1/2/3 to enable MIP-cutting planes in MISLP
 - Generally speeds up the solve
 - May lead to local infeasibility by cutting off bad local optima
 - **slpheurstrategy**: Controls heuristics in MISLP
 - When left at default, also takes **heuremphasis** into account

Guidelines for solver choice and tuning

- When to use Global solver:
 - Whenever a reliable bound on the optimal value is needed
 - Problems with many local optima
 - Infeasibility detection or repair

Guidelines for solver choice and tuning

- When to use Global solver:
 - Whenever a reliable bound on the optimal value is needed
 - Problems with many local optima
 - Infeasibility detection or repair
- When to use SLP over Global:
 - When quick solves are crucial (Global can be orders of magnitude slower, especially for NLPs)
 - Large-scale problems where Global's branch-and-bound becomes prohibitive
 - Problems with user functions (not supported by Global)

Guidelines for solver choice and tuning

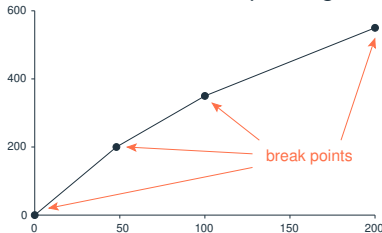
- **When to use Global solver:**
 - Whenever a reliable bound on the optimal value is needed
 - Problems with many local optima
 - Infeasibility detection or repair
- **When to use SLP over Global:**
 - When quick solves are crucial (Global can be orders of magnitude slower, especially for NLPs)
 - Large-scale problems where Global's branch-and-bound becomes prohibitive
 - Problems with user functions (not supported by Global)
- **When to use Knitro:**
 - Highly nonlinear models
 - When SLP convergence is slow

Guidelines for solver choice and tuning

- **When to use Global solver:**
 - Whenever a reliable bound on the optimal value is needed
 - Problems with many local optima
 - Infeasibility detection or repair
- **When to use SLP over Global:**
 - When quick solves are crucial (Global can be orders of magnitude slower, especially for NLPs)
 - Large-scale problems where Global's branch-and-bound becomes prohibitive
 - Problems with user functions (not supported by Global)
- **When to use Knitro:**
 - Highly nonlinear models
 - When SLP convergence is slow
- **General tuning strategy:**
 - Start with default settings
 - Use Xpress Tuner to automatically find good parameter settings (be careful with tuner for SLP/local solves – may optimize for speed over solution quality)
 - Fine-tune specific controls based on problem characteristics

Piecewise linear (PWL) functions

- Piecewise linear constraints define a variable as a **piecewise linear function of another variable**:
 - Can be used to **model stepwise functions** or to **approximate nonlinear functions**
 - Example for discounts on unit costs depending on the quantity of items bought:



- First 50 items: $COST_1 = \$4$ each
 - Next 50 items: $COST_2 = \$3$ each
 - Then, up to 200: $COST_3 = \$2$ each
- Quantity break points x_i : 0, 50, 100, 200
 - Cost break points y_i (= total cost of buying quantity x_i): 0, 200, 350, 550
 $y_i = COST_i \cdot (x_i - x_{i-1}) + y_{i-1}$ for $i = 1, 2, 3$

Piecewise linear (PWL) functions

- Piecewise linear functions can be intuitively added to a problem by using the `xp.pwl(dict)` method in constraints or objectives:
 - Receives a **dictionary** as argument that associates intervals with linear functions with:
 - **Keys**: intervals specified as **two-element tuples** (must be pairwise disjoint, no overlap)
 - **Values**: linear expressions (or constants). Function must use only one variable in all of the dictionary's values
 - Modeling the previous example where y is a piecewise linear function of x :

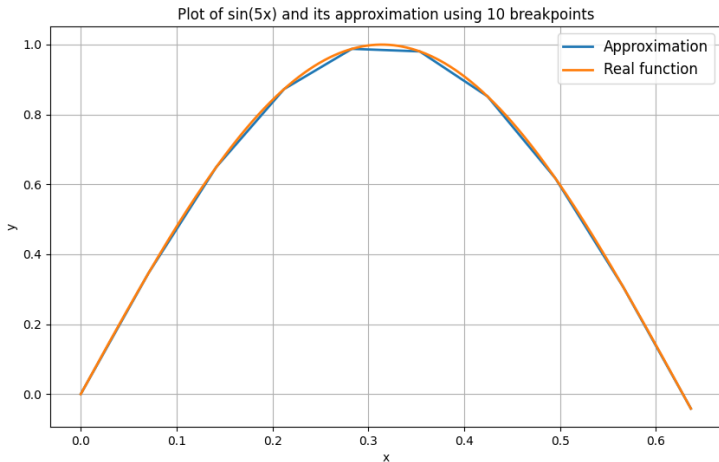
```
x = p.addVariable(vartype=xp.integer, ub=200)
y = p.addVariable()
p.addConstraint(xp.pwl({(0, 50): 4*x,
                       (50, 100): 3*(x-50) + 200,
                       (100, 200): 2*(x-100) + 350})) == y)
```



Note: *The piecewise linear function is always univariate, i.e. there must always be only one input variable*

Piecewise linear approximation of nonlinear functions

- Piecewise linear constraints can also be used to approximate nonlinear functions:
 - Approximating the function $\sin(5x)$, $\forall x$ in $[0, 2/\pi]$, to be maximized:



Piecewise linear approximation using `p.addpwlcons`

- The low-level function `problem.addPWLCns` can be used for creating and directly adding piecewise linear constraints to a problem:

```
problem.addPWLCns(colind, resultant, start, xval, yval)
```

- `colind`: integer array containing the input variables x of the piecewise linear functions
- `resultant`: integer array containing the output variables y of the piecewise linear functions
- `start`: integer array containing the start index of each constraint in the `xval` and `yval` arrays
- `xval`: array containing the x -values of the breakpoints
- `yval`: array containing the y -values of the breakpoints



Note: A PWL constraint $y = f(x)$ consists of an (input) column x , a resultant (output) column y and a piecewise linear function f described by breakpoints $(xval, yval)$. The function does not need to be continuous - if there are jumps, the solver may return values at any point in the jump

Piecewise linear approximation using `p.addpwlcons`

1. Define breakpoints and values at breakpoints:

```
N = 10          # number of segments
freq = 5        # frequency
step = (2 / math.pi) / (N - 1)  # width of each segment = domain / (N-1)
breakpoints = np.array([i * step for i in range(N)]) # breakpoints x
values = np.sin(freq * breakpoints)                # value at breakpoints
```

2. Define piecewise linear function using `p.addPWLCons`:

```
y = p.addVariable()          # auxiliary variable for 'resultant'

p.addPWLCons([x], [y], [0], breakpoints, values)
```

3. Set `y` as the objective function to be maximized, and optimize:

```
p.setObjective(y, xp.ObjSense.MAXIMIZE)

p.optimize()
```

Piecewise linear approximation using `xp.pwl`

1. Define breakpoints, values at breakpoints and `slopes`:

```
N = 10          # number of segments
freq = 5        # frequency
step = (2 / math.pi) / (N - 1)  # width of each segment = domain / (N-1)
breakpoints = np.array([i * step for i in range(N)]) # breakpoints x
values = np.sin(freq * breakpoints)                # value at breakpoints
slopes = freq * np.cos(freq * breakpoints)          # derivatives
```

2. Define piecewise linear function using `list comprehension` with `xp.pwl`:

```
pw = xp.pwl((breakpoints[i], breakpoints[i+1]):
            values[i] + slopes[i] * (x - breakpoints[i]) for i in range(N - 1))
```

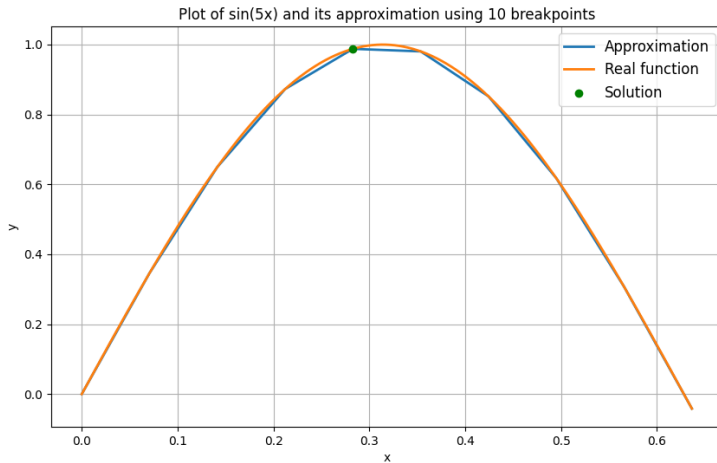
3. Set `pw` as the objective function to be maximized, and optimize:

```
p.setObjective(pw, xp.ObjSense.MAXIMIZE)

p.optimize()
```

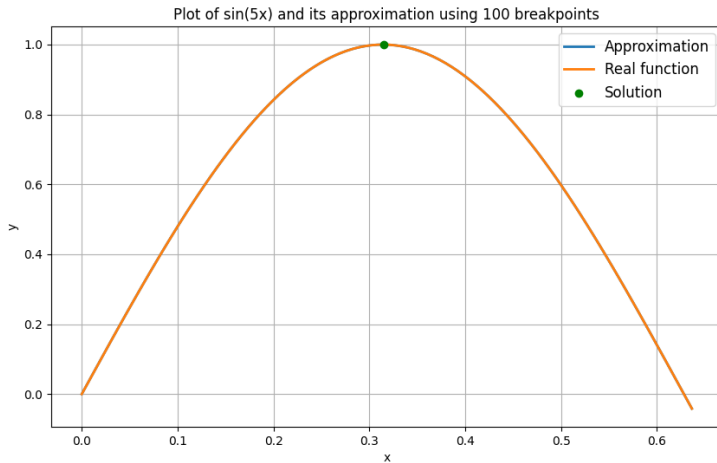
Piecewise linear approximation

- Solution for $N = 10$:



Piecewise linear approximation

- Solution for $N = 100$:



Python Notebook [PN-4.2]: Using PWL to approximate a nonlinear function

- Navigate to [Exercise 4.2 - Using PWL to approximate a nonlinear function](#)
- Run the code cells in [Exercise 4.2](#) for progressively higher number of breakpoints N : 10, 15, 20, ...
- Visualize the evolution of the piecewise linear approximation and the approximation of the nonlinear function



Thank You

www.fico.com/optimization